

Domain 2 — Section Review



Tool Design & MCP Integration

7/7

100% · Strong · 0 missed

Q1 ✓ CORRECT



Production logs show the agent frequently calls `get_customer` when users ask about orders (e.g., "check my order #12345"), instead of `lookup_order`. Both tools have minimal descriptions ("Retrieves customer information" / "Retrieves order details") and accept similar identifier formats. What is the most effective first step to improve tool selection reliability?

- A Add few-shot examples to the system prompt demonstrating correct tool selection patterns, with 5–8 examples showing order-related queries routing to `lookup_order`.
- B Expand each tool's description to include input formats it handles, example queries, edge cases, and boundaries explaining when to use it versus similar tools.**
- C Implement a routing layer that parses user input before each turn and pre-selects the appropriate tool based on detected keywords.
- D Consolidate both tools into a single `lookup_entity` tool that accepts any identifier and internally determines which backend to query.

Why B is correct:

Tool descriptions ARE the routing mechanism. Fix at the cheapest layer that addresses the root cause. Expanding descriptions is the highest-leverage first step — low effort, targets root cause directly. Few-shot examples (A) treat the symptom with added token overhead. A routing layer (C) is over-engineered. Consolidation (D) is a valid architecture but far more costly than a description edit.

Q2 ✓ CORRECT

Production logs reveal a consistent pattern: requests to "analyse the quarterly report I uploaded" are routed to the web search agent 45% of the time instead of the document analysis agent. Examining the tool definitions, you find the web search agent has an `analyze_content` tool described as "analyzes content and extracts key information," while the document analysis agent has an `analyze_document` tool described as "analyzes documents and extracts key information." How should you address this misrouting?

- A** Add few-shot examples to the system prompt showing document analysis queries routed to the document analysis agent.  
- B** Rename `analyze_content` to `extract_web_results` and update its description to specify it handles web search results; update `analyze_document` to clarify it handles uploaded files, PDFs, and local documents.
- C** Merge `analyze_content` and `analyze_document` into a single tool so the routing decision is eliminated.
- D** Add a system prompt instruction: "Always route document upload requests to the document analysis agent."

Why B is correct:

Rename + describe to eliminate functional overlap. The near-identical descriptions ("analyzes content" vs "analyzes documents") are the root cause. The fix: rename `analyze_content` to `extract_web_results` and update both descriptions to specify input types, example queries, and explicit boundaries. Keyword instructions in the system prompt (D) compete with tool descriptions rather than fixing them. Merging (C) loses useful specialisation.

Q3 ✓ **CORRECT**

After expanding tool descriptions to fix misrouting, 30% of requests still route to the wrong tool. What should you investigate next?

- A** Add 10–15 more few-shot examples until routing accuracy improves past 95%.
- B** Audit the system prompt for keyword-sensitive instructions that create unintended tool associations — overriding the improved descriptions.

C Merge the two tools to eliminate the routing decision entirely.



D Deploy a dedicated routing classifier as a pre-processing step.

Why B is correct:

The system prompt keyword trap. Even with improved tool descriptions, a system prompt instruction like "Always look up the customer first" creates a keyword-tool association that overrides the descriptions. After fixing descriptions, audit the system prompt for conflicting instructions. The fix is to remove those associations — not add more routing rules that compete with descriptions.

Q4 ✓ **CORRECT**

During testing, you observe that the synthesis agent frequently needs to verify specific claims while combining findings. Currently, when verification is needed, the synthesis agent returns control to the coordinator, which invokes the web search agent, then re-invokes synthesis with results. This adds 2–3 round trips per task and increases latency by 40%. Your evaluation shows that 85% of these verifications are simple fact-checks (dates, names, statistics) while 15% require deeper investigation. What is the most effective approach to reduce overhead while maintaining system reliability?

A Give the synthesis agent access to all web search tools so it can handle any verification need directly without round-trips through the coordinator.

B Have the synthesis agent accumulate all verification needs and return them as a batch to the coordinator at the end of its pass, which then sends them all to the web search agent at once.

C Give the synthesis agent a scoped `verify_fact` tool for simple lookups, while complex verifications continue delegating to the web search agent through the coordinator.

D Have the web search agent proactively cache extra context around each source during initial research, anticipating what the synthesis agent might need to verify.

Why C is correct:

Principle of least privilege + proportionate fix. Give synthesis only what it needs for the 85% simple case via a scoped tool. The existing coordination pattern handles the 15% complex cases. Giving all web search tools (A) violates separation of concerns. Batching (B) creates blocking dependencies — synthesis steps may depend on earlier verified facts. Speculative caching (D) cannot reliably predict what will be needed.

Q5 ✓ **CORRECT**

An MCP tool returns `{"isError": false, "results": []}` when no papers are found. A different tool returns `{"isError": true, "message": "Database timeout"}` on a connectivity failure. A teammate proposes unifying both to `{"status": "no_results"}`. Why is this problematic?

- A The isError flag is required by the MCP specification and cannot be omitted.
- B Conflating an access failure with a valid empty result prevents the coordinator from knowing whether to retry — destroying recovery logic.**
- C Agents always retry on isError:true, so unification would cause unnecessary retry loops.
- D The status field is not a recognised MCP response format.

Why B is correct:

isError:true means 'couldn't look.' **isError:false + empty means 'looked, found nothing.'** These require opposite coordinator responses: access failures → retry or escalate; valid empty results → continue with alternate approaches. Unifying them makes both look identical, destroying the recovery decision signal entirely.

Q6 ✓ CORRECT



You want to share an MCP server configuration with your entire team via version control, using an environment variable for the auth token. Correct configuration:

- A `~/.claude.json` with the token hardcoded, distributed via onboarding documentation.
- B `.mcp.json` in the project repository with `${GITHUB_TOKEN}` environment variable expansion.**
- C `~/.claude/CLAUDE.md` with an `@import` pointing to the MCP server config.
- D `.claude/settings.json` with the token stored as a CI/CD pipeline secret.

Why B is correct:

Project-level MCP = `.mcp.json` in the repo root. Version-controlled, available to all team members on clone/pull. Environment variable expansion (e.g., `${GITHUB_TOKEN}`) handles credentials without committing secrets. `~/.claude.json` is for personal/experimental servers — not shared via version control.

Q7 ✓ CORRECT

While helping a developer refactor a large utility file, Claude attempts to use the Edit tool to update a repeated helper pattern, but the edit fails — the anchor text appears dozens of times and is not unique. What's the correct recovery approach?

- A Use Grep to find all instances, then run Edit once per match using the surrounding lines as unique anchor text.
- B Use Bash with `sed` to perform a global search-and-replace across the file.**

- C** Use Read to load the full file contents, apply all modifications, then use Write to write the complete updated file.   

- D** Use Glob to identify all files containing the pattern and edit them one at a time.

Why C is correct:

When Edit fails due to non-unique anchor text, the fallback is Read + Write. Load the full file, apply all changes in memory, write it back completely. This is reliable regardless of how many times the pattern appears. Multiple Edit calls (A) still fail when the surrounding context is also repeated. sed via Bash (B) works but bypasses the intended tooling. Glob (D) matches file paths by pattern — it's not a content-modification tool.

Next Section →